# Unix & Shell Programming

Module 3 – File Content Manipulation Commands

Question & Answer Notes

BCA Semester 6 · MAKAUT

## Q1. Selecting Rows and Columns from a File

UNIX provides several commands to extract specific rows (lines) and columns from a file.

**Selecting Rows** – `head`, `tail`, `grep`, `sed`

```
head -5 file.txt            # show first 5 lines
head -n 10 file.txt         # show first 10 lines

tail -5 file.txt            # show last 5 lines
tail -n 10 file.txt         # show last 10 lines
tail -f logfile.txt         # follow live updates (great for
   logs)

grep "error" file.txt       # show all lines containing "error"
grep -n "hello" file.txt    # show matching lines with line
   numbers
grep -v "error" file.txt    # show all lines NOT containing "
   error"
grep -i "error" file.txt    # case-insensitive search

sed -n '3p' file.txt        # print only line 3
sed -n '3,7p' file.txt      # print lines 3 to 7
sed -n '1~2p' file.txt      # print every alternate line (1, 3,
   5...)
```

**Selecting Columns** – `cut`

The `cut` command extracts specific columns (fields) from each line.

**Syntax:** `cut [options] file`

```
cut -c1-5 file.txt          # extract characters 1 to 5 from
   each line
cut -c1,3,5 file.txt        # extract characters at positions 1,
    3, and 5

cut -d":" -f1 /etc/passwd   # delimiter=colon, get field 1 (
   usernames)
cut -d":" -f1,3 /etc/passwd # get fields 1 and 3
cut -d"," -f2 data.csv      # get 2nd column from CSV file
cut -d" " -f1-3 file.txt    # get first 3 space-separated fields
```

**Example – suppose `data.txt` contains:**

```
Alice:25:Kolkata
Bob:30:Delhi
Carol:22:Mumbai
```

```
cut -d":" -f1 data.txt      # Output: Alice / Bob / Carol
cut -d":" -f2 data.txt      # Output: 25 / 30 / 22
cut -d":" -f1,3 data.txt    # Output: Alice:Kolkata / Bob:Delhi
```

**Using `awk` for Rows and Columns Together:**

```
awk '{print $1}' file.txt            # print first column (space
   -separated)
awk '{print $1, $3}' file.txt        # print columns 1 and 3
awk -F":" '{print $1}' /etc/passwd   # colon delimiter, print
   first field
awk 'NR==3' file.txt                 # print only line 3
awk 'NR>=2 && NR<=5' file.txt        # print lines 2 to 5
awk '$2 > 25' data.txt               # print rows where column 2
    > 25
```

**Summary:** Use `head`/`tail`/`grep`/`sed` for rows. Use `cut`/`awk` for columns. `awk` is the most powerful – it can do both.

## Q2. Merging Lines of Files Horizontally – `paste`

The `paste` command merges files **side by side** (horizontally), line by line.

**Syntax:** `paste [options] file1 file2 ...`

**Example files:**

```
# names.txt          # scores.txt
Alice                95
Bob                  80
Carol                72
```

```
paste names.txt scores.txt
# Output:
# Alice    95
# Bob      80
# Carol    72
```

**paste options:**

```
paste -d"," names.txt scores.txt
# Use comma as delimiter instead of tab
# Output: Alice,95 / Bob,80 / Carol,72
```

```
paste -d":" file1 file2 file3
# Merge 3 files with colon as separator

paste -s names.txt
# Merge all lines of ONE file into a single line (serial mode)
# Output: Alice    Bob    Carol

paste -s -d"," names.txt
# Serial with comma: Alice,Bob,Carol
```

**Comparison – `cat` vs `paste`:**

| Command | Direction | Effect |
|---|---|---|
| `cat file1 file2` | Vertical (top to bottom) | Appends file2 below file1 |
| `paste file1 file2` | Horizontal (side by side) | Merges files column by column |

## Q3. Splitting Large Files into Smaller Files – `split`

The `split` command divides a large file into smaller pieces.

**Syntax:** `split [options] inputfile [prefix]`

**Split by number of lines (default):**

```
split bigfile.txt                    # split into 1000-line
   chunks (default)
# Creates: xaa, xab, xac, ...

split -l 500 bigfile.txt             # split into 500-line chunks
split -l 500 bigfile.txt part_       # name chunks: part_aa,
   part_ab, ...
```

**Split by file size:**

```
split -b 1M bigfile.txt              # split into 1 MB pieces
split -b 500k bigfile.txt chunk_     # split into 500 KB pieces
   named chunk_aa...
split -b 1G largefile.iso parts_     # split a 1 GB file
```

**Split into N equal parts:**

```
split -n 4 bigfile.txt               # split into exactly 4 equal
    parts
```

**Numeric suffixes:**

```
split -l 100 -d bigfile.txt part_
# Creates: part_00, part_01, part_02 ... (numeric instead of aa
   , ab)
```

**Reassemble split files:**

```
cat part_* > restored_file.txt      # combine all parts back
    into one file
```

**Use case:** Splitting is useful when emailing large log files, uploading to limited-size storage, or processing large data files in chunks.

## Q4. Counting Lines, Words, and Characters – `wc`

The `wc` (word count) command counts lines, words, and characters in a file.

**Syntax:** `wc [options] filename`

```
wc file.txt
# Output: 10  45  230  file.txt
#           ^   ^    ^
#         lines words characters
```

**Options:**

| Option | Description |
|---|---|
| `wc -l file` | Count only lines |
| `wc -w file` | Count only words |
| `wc -c file` | Count only bytes (characters) |
| `wc -m file` | Count only characters (handles multibyte/Unicode) |
| `wc -L file` | Show length of the longest line |

```
wc -l file.txt                  # Output: 10 file.txt
wc -w file.txt                  # Output: 45 file.txt
wc -c file.txt                  # Output: 230 file.txt

wc -l *.txt                     # count lines in all .txt files
ls | wc -l                      # count number of files in current
    directory
cat file.txt | wc -w            # count words using pipe
grep "error" log.txt | wc -l # count how many lines contain "
    error"
```

**Multiple files:**

```
wc file1.txt file2.txt
# Output:
#  10  45  230  file1.txt
#   5  20  100  file2.txt
#  15  65  330  total
```

## Q5. Sorting a File by a Specific Column – `sort`

The `sort` command arranges lines of a file in a specified order.

**Syntax:** `sort [options] file`

**Basic sorting:**

```
sort file.txt                # alphabetical sort (ascending, default)
sort -r file.txt             # reverse order (descending)
sort -u file.txt             # sort and remove duplicate lines
```

**Sorting by a specific column:**

Use `-k` (key) option to specify which column to sort by.

**Example** – `students.txt`:

```
Alice   Chemistry   72
Bob     Maths       95
Carol   Physics     80
Dave    Maths       65
```

```
sort -k1 students.txt        # sort by column 1 (name) -- alphabetical
sort -k2 students.txt        # sort by column 2 (subject)
sort -k3 students.txt        # sort by column 3 (marks) -- but as text!
sort -k3 -n students.txt     # sort by column 3 numerically (correct)
sort -k3 -nr students.txt    # sort by marks, highest first
```

**Sorting CSV or colon-separated files:**

```
sort -t":" -k3 -n /etc/passwd   # sort passwd by UID (field 3), numeric
sort -t"," -k2 data.csv         # sort CSV by column 2
```

**Common sort options:**

| Option | Description |
|---|---|
| `-k N` | Sort by column N |
| `-t DELIM` | Use DELIM as field separator (default: whitespace) |
| `-n` | Numeric sort (treat values as numbers) |
| `-r` | Reverse sort order |
| `-u` | Unique – remove duplicates after sorting |
| `-f` | Ignore case (fold uppercase to lowercase) |
| `-b` | Ignore leading blank spaces |

## Q6. Numerical Sorting

By default, `sort` sorts **alphabetically** (as text). This means 10 comes before 2 because "1" ¡ "2" in text comparison.

**Numerical sort** (`-n`) treats values as actual numbers.

**Example** – `numbers.txt`:

```
100
5
23
8
1000
42
```

```
sort numbers.txt
# Text sort output (wrong order for numbers):
# 100
# 1000
# 23
# 42
# 5
# 8

sort -n numbers.txt
# Numeric sort output (correct order):
# 5
# 8
# 23
# 42
# 100
# 1000
```

**More numerical sort examples:**

```
sort -n -r numbers.txt        # descending numeric: 1000, 100,
   42, 23, 8, 5

ls -s | sort -n               # sort files by size (smallest
   first)
ls -s | sort -n -r            # sort files by size (largest
   first)

du -h * | sort -h             # human-readable size sort (1K, 5M
   , 2G sorted correctly)

sort -k2 -n students.txt      # numerically sort by 2nd column
```

**Remember:** Always use `-n` when sorting numbers. Without it, 10 appears before 2, which is wrong. Use `-h` for human-readable sizes like 1K, 2M, 3G.

## Q7. Comparing Two Files Line by Line – `diff`

The `diff` command compares two files and shows the **lines that are different**.

**Syntax:** `diff file1 file2`

**Example files:**

```
# file1.txt              # file2.txt
Hello                    Hello
World                    Universe
UNIX                     UNIX
is great                 is great
Bye                      Goodbye
```

```
diff file1.txt file2.txt
# Output:
# 2c2
# < World
# ---
# > Universe
# 5c5
# < Bye
# ---
# > Goodbye
```

**Understanding diff output symbols:**

| Symbol | Meaning |
|---|---|
| `<` | Line from file1 (left file) |
| `>` | Line from file2 (right file) |
| `---` | Separator between the two differing sections |
| `Nc` | Line N was Changed |
| `Na` | Line was Added (in file2) |
| `Nd` | Line was Deleted (from file1) |

**diff options:**

```
diff -i file1 file2          # ignore case differences
diff -w file1 file2          # ignore whitespace differences
diff -u file1 file2          # unified format (easier to read,
   used in patches)
diff -y file1 file2          # side-by-side comparison
diff -q file1 file2          # only say if files differ (no
   details)
```

```
diff -r dir1/ dir2/          # compare two directories
   recursively
```

## Q8. Comparing Two Files Character by Character – cmp

The cmp command compares two files **byte by byte** (character by character) and reports the first difference found.

**Syntax:** cmp [options] file1 file2

```
cmp file1.txt file2.txt
# If different:
# file1.txt file2.txt differ: byte 7, line 2

# If same:
# (no output -- files are identical)
```

**cmp options:**

```
cmp -l file1.txt file2.txt
# List ALL bytes that differ (byte offset + values in octal)
# Output: 7  127  125   (byte 7, value in file1, value in file2
   )

cmp -s file1.txt file2.txt
# Silent mode -- no output; use exit code in scripts
# Exit code 0 = files are same
# Exit code 1 = files are different

cmp -i 5 file1.txt file2.txt
# Skip first 5 bytes, then compare
```

**Using cmp in a shell script:**

```
cmp -s file1.txt file2.txt
if [ $? -eq 0 ]; then
    echo "Files are identical"
else
    echo "Files are different"
fi
```

**diff vs cmp – comparison:**

| Feature | `diff` | `cmp` |
|---|---|---|
| **Comparison unit** | Line by line | Byte by byte |
| **Works on** | Text files | Any file (text or binary) |
| **Shows all differences** | Yes | Only first difference |
| **Output format** | Lines added/changed/deleted | Byte offset and line number |
| **Best for** | Source code, config files | Binary files, quick check |

## Q9. Comparing Two Sorted Files – `comm`

The `comm` command compares two **already-sorted** files and shows lines that are:

- Only in file1
- Only in file2
- In both files

**Syntax:** `comm file1 file2`

**Important:** Both files must be sorted first.

**Example:**

```
# sorted1.txt          # sorted2.txt
apple                  apple
banana                 cherry
cherry                 grape
mango                  mango
```

```
comm sorted1.txt sorted2.txt
# Output (3 columns):
#            apple                <- in both
# banana                          <- only in file1
#                      cherry     <- only in file2
#            cherry                <- wait, let me re-sort...

# Column 1: only in file1
# Column 2: only in file2
# Column 3: in both files
```

**Suppressing columns with -1, -2, -3:**

```
comm -12 file1 file2    # show only common lines (suppress col
   1 and 2)
comm -23 file1 file2    # show only lines unique to file1
comm -13 file1 file2    # show only lines unique to file2
comm -3  file1 file2    # hide common lines, show unique lines
   from each
```

**Practical use:**

```
sort list1.txt > s1.txt
sort list2.txt > s2.txt
comm -12 s1.txt s2.txt    # find common items between two lists
comm -23 s1.txt s2.txt    # find items only in list1 (not in
   list2)
```

**Key rule:** Files must be sorted before using `comm`. Use `sort` first, then `comm`.

## Q10. The `join` Command

The `join` command merges two files based on a **common field** (like a database join). Both files must be sorted on the join field.

**Syntax:** `join [options] file1 file2`

**Example:**

```
# employees.txt        # salaries.txt
101 Alice              101 50000
102 Bob                103 70000
103 Carol              104 60000
```

```
join employees.txt salaries.txt
# Output (joins on field 1 = employee ID):
# 101 Alice 50000
# 103 Carol 70000
# (102 Bob has no salary entry, so excluded by default)
```

**join options:**

```
join -1 2 -2 1 file1 file2
# Join on field 2 of file1 and field 1 of file2

join -a 1 file1 file2
# Include unmatched lines from file1 (like LEFT JOIN in SQL)

join -a 2 file1 file2
# Include unmatched lines from file2 (like RIGHT JOIN in SQL)

join -a 1 -a 2 file1 file2
# Include all unmatched lines from both (like FULL OUTER JOIN)

join -t":" -1 1 -2 1 file1 file2
# Use colon as delimiter

join -o 1.1,1.2,2.2 file1 file2
```

```
# Custom output format: field1 of file1, field2 of file1,
    field2 of file2
```

**Comparison – join vs paste:**

| Feature | join | paste |
|---|---|---|
| **Merge method** | Based on common field | Line by line (positional) |
| **Files must match** | Only matching rows merged | All rows merged regardless |
| **Like SQL** | Yes (like JOIN) | No |
| **Sorted input required** | Yes | No |

## Q11. The uniq Command

The `uniq` command removes or reports **duplicate adjacent lines** from a file.

**Syntax:** `uniq [options] [input] [output]`

**Important:** `uniq` only removes *adjacent* (consecutive) duplicates. So the file must usually be sorted first.

**Example** – `fruits.txt`:

```
apple
apple
banana
cherry
cherry
cherry
banana
```

```
uniq fruits.txt
# Output:
# apple
# banana
# cherry
# banana      <- NOT removed! (not adjacent to earlier 'banana')

sort fruits.txt | uniq
# Output:
# apple
# banana
# cherry      <- All duplicates removed after sorting
```

**uniq options:**

| Option | Description |
|---|---|
| `uniq -c file` | Count how many times each line appears (prefix with count) |
| `uniq -d file` | Show only lines that are DUPLICATED |
| `uniq -u file` | Show only lines that appear exactly ONCE (unique) |
| `uniq -i file` | Ignore case when comparing |
| `uniq -f N file` | Skip first N fields before comparing |
| `uniq -s N file` | Skip first N characters before comparing |

```
sort fruits.txt | uniq -c
# Output:
#    1 apple
#    2 banana
#    3 cherry

sort fruits.txt | uniq -d
# Output (only duplicated lines):
# banana
# cherry

sort fruits.txt | uniq -u
# Output (lines appearing only once):
# apple

sort fruits.txt | uniq -c | sort -rn
# Count + sort by frequency (most common first)
```

**Common pattern:** `sort file | uniq -c | sort -rn` – Sort a file, count occurrences, then sort by most frequent. Very useful for log analysis.

## Q12. The `tr` (Translation/Transformation) Command

The `tr` command **translates, deletes, or squeezes** characters from standard input.

**Syntax:** `tr [options] SET1 [SET2]`

`tr` does NOT take a filename – input must come via pipe or redirection.

**Basic Translation (Replace characters):**

```
echo "hello" | tr 'a-z' 'A-Z'
# Output: HELLO   (lowercase to uppercase)

echo "HELLO WORLD" | tr 'A-Z' 'a-z'
# Output: hello world   (uppercase to lowercase)

echo "hello123" | tr '0-9' '#'
# Output: hello###   (replace digits with #)

echo "hello world" | tr ' ' '_'
```

```
# Output: hello_world   (replace spaces with underscore)

cat file.txt | tr 'a-z' 'A-Z'    # convert entire file to
   uppercase
```

**Deleting Characters (`-d`):**

```
echo "hello 123 world" | tr -d '0-9'
# Output: hello  world   (removes all digits)

echo "hello, world!" | tr -d '[:punct:]'
# Output: hello world   (removes all punctuation)

echo "Hello World" | tr -d ' '
# Output: HelloWorld   (removes spaces)

cat file.txt | tr -d '\r'
# Remove Windows carriage return characters (fix Windows text
   files on UNIX)
```

**Squeezing Repeated Characters (`-s`):**

```
echo "heeello    world" | tr -s 'e'
# Output: hello    world   (multiple 'e' squeezed to one)

echo "hello      world" | tr -s ' '
# Output: hello world   (multiple spaces squeezed to one)

cat file.txt | tr -s '\n'
# Squeeze multiple blank lines into one
```

**Complement (`-c`) – work on all characters NOT in SET1:**

```
echo "hello 123 world" | tr -cd '0-9'
# Output: 123   (keep only digits, delete everything else)

echo "hello world" | tr -cd 'a-zA-Z\n'
# Keep only letters and newlines (remove numbers, punctuation,
   etc.)
```

**Character Classes in `tr`:**

| Class | Matches |
|---|---|
| [:lower:] | Lowercase letters (a-z) |
| [:upper:] | Uppercase letters (A-Z) |
| [:alpha:] | All letters (a-z and A-Z) |
| [:digit:] | Digits (0-9) |
| [:alnum:] | Letters and digits |
| [:space:] | Whitespace (space, tab, newline) |
| [:punct:] | Punctuation characters |
| [:print:] | Printable characters |

```
echo "Hello World 123!" | tr '[:lower:]' '[:upper:]'
# Output: HELLO WORLD 123!

echo "abc123" | tr -d '[:digit:]'
# Output: abc
```

**Summary of tr options:**

| Option | Description |
|---|---|
| (none) | Translate: replace characters in SET1 with SET2 |
| -d SET1 | Delete all characters in SET1 |
| -s SET1 | Squeeze repeated characters in SET1 into one |
| -c SET1 | Complement: apply operation to all chars NOT in SET1 |
| -cd SET1 | Delete all characters NOT in SET1 (keep only SET1) |

**Remember:** `tr` reads from stdin only – always use a pipe (`|`) or input redirection (`<`) with it. Example: `cat file.txt | tr 'a-z' 'A-Z'` or `tr 'a-z' 'A-Z' < file.txt`

— End of Module 3 Notes —